

OpenMower-TAF-App

GPS-State MQTT-Vertragsintegration

State1 bis State4, Lease-Steuerung, F9P-Neustart, Einstellungen und GPS-Logging

Angabe	Wert
Version	v0.16
Datum	15.07.2026
Ausgangspaket	OpenMower-TAF-App (4)(2).zip
Schnittstellengrundlage	OpenMower-TAF-ROS MQTT-Schnittstellenvertrag GPS-State App-Integration v1.0
Status	Implementiert; statische Quelltextprüfung durchgeführt

1. Ziel

Die GPS-State-Seite wurde auf die kanonische MQTT-Schnittstelle des ROS-Pakets ausgerichtet. Verwendet werden ausschließlich State1 bis State4. Dynamische Detailzustände werden nur bei geöffneter Ansicht über Client-Leases aktiviert. Restart, Einstellungen und Logger folgen den im Vertrag definierten Topics und Zustandsmodellen.

2. Umgesetzte MQTT-Anpassungen

Bereich	Umsetzung
Legacy-States	Abonnements und Verarbeitung der MQTT-Topics state0/definition und state0/status entfernt.
State1	Fahrfähigkeitsdialog nutzt state1/definition und state1/status. Manuelle Aktualisierung sendet publish_now an state1/request.
State2	state2/status und state2/satellites werden verarbeitet. Öffnen der Leiste aktiviert eine Lease mit 500 ms Intervall.
State3	Beim Öffnen der State2-Satellitenansicht wird die Liste verwendeter Satelliten per Lease aktiviert; Intervall 1000 ms.
State4	Vollständige sichtbare Satellitenliste wird nur über explizite Aktivierung zugeschaltet und beim Schließen deaktiviert.
Lease	Lease 15000 ms; Erneuerung alle 10 Sekunden; stabile request_id je App-Instanz und State.
Zentrale Erneuerung	Bleibt für Definitionen, Settings und einmalige Snapshots erhalten; aktiviert keine geschlossene Detailansicht dauerhaft.

3. Fahrfähigkeitsdialog

Die intern vorhandene Tabellenlogik wird nun aus dem kanonischen State1 gespeist. Dadurch bleiben die bestehende zwölfstufige Darstellung und die Markierung des ersten Blockierers erhalten, ohne Legacy-Topics zu verwenden. Alle vom Backend gelieferten Checks werden angezeigt; drive_ready wird nicht lokal neu berechnet.

Die Schaltfläche wurde in „Fahrfähigkeit aktualisieren“ umbenannt. Der Request enthält command=publish_now und eine request_id.

4. Bedarfsgesteuerte Detailansichten

App-Ereignis	MQTT-Verhalten
State2-Leiste öffnen	set_active, active=true für State2 und State3; sofortiger Snapshot und zyklische Daten.
Leiste öffnen	Lease-Erneuerung vor Ablauf.
Alle sichtbaren Satelliten einschalten	set_active, active=true für State4.
Leiste schließen	active=false für State2, State3 und gegebenenfalls State4.

Controller wird beendet	Aktive Leases werden explizit beendet; Backend-Timeout bleibt zusätzliche Absicherung.
-------------------------	--

5. Erweiterte F9P-Neustartfunktionen

Die Neustartmodi `hot_start`, `warm_start` und `cold_start` bleiben erhalten. Die Reset-Modi entsprechen nun dem Vertrag: `controlled_software`, `gnss_only`, `hardware_watchdog` und `hardware_after_shutdown`. Jeder Befehl enthält eine `request_id`. Während eines laufenden Neustarts bleiben weitere Neustartkommandos gesperrt; `status`, `last` und `validation` bleiben getrennte Quellen.

6. GPS-State-Einstellungen

Der obsolete Schlüssel `publish_state0` wurde aus der App-seitigen Schlüsselliste entfernt. `State2` bis `State4` werden nicht mehr über persistente `publish`-Schalter gesteuert, sondern über die Laufzeit-Leases. Session- und persistente Änderungen sowie Validierungsantworten bleiben getrennt. Logging-Vorgaben werden weiterhin dynamisch aus `gps_state/settings/json` bezogen.

7. GPS-Logging

Die Laufzeitsteuerung bleibt auf `gps_state/logging/set/control/json`. `Start`, `Stop` und `Cancel` werden weiterhin ausschließlich aus `logging/status/json` abgeleitet. Die App ergänzt bei Steuerkommandos automatisch eine `request_id`, falls der erzeugte Payload noch keine enthält. Der bestehende Logging-v2-Schemacheck und die getrennte Anzeige von `logging/last/json` bleiben erhalten.

8. Geänderte Dateien

Datei	Änderung
<code>lib/io/mqtt_connection.dart</code>	Kanonische Topics, <code>State2</code> -Satellitenliste, <code>Request-Publisher</code> , <code>Legacy-State0</code> entfernt.
<code>lib/controllers/gps_state_controller.dart</code>	<code>State1</code> -Zuordnung, Lease-Verwaltung, Restart-Modi, Request-IDs und Export <code>State1-State4</code> .
<code>lib/controllers/satellite_logging_controller.dart</code>	Request-ID für Logging-Kommandos.
<code>lib/screens/gps_state.dart</code>	Öffnungsabhängige Aktivierung, neue Texte, <code>State1</code> -Aktualisierung und erweiterter Reset-Modus.

9. Prüfung

Prüfung	Ergebnis
<code>git diff --check</code>	Ohne Whitespace-Fehler.
Legacy-Topic-Prüfung	<code>state0</code> MQTT-Konstanten, Abonnements und Parserfälle entfernt.
Klammerprüfung	Geänderte Dart-Dateien mit ausgeglichenen geschweiften und runden Klammern.
Flutter/Dart Build	In der Ausführungsumgebung war kein Flutter-/Dart-SDK vorhanden; ein vollständiger Build konnte daher nicht ausgeführt werden.

10. Abnahmetests im Zielsystem

- GPS-State-Seite öffnen: `State1` erscheint aus `retained state1/status`; `State2` bis `State4` senden noch keine zyklischen Daten.
- `State2` aufklappen: Requests für `State2` und `State3` mit `active=true`; Zeitstempel steigen regelmäßig.
- `State2` zuklappen: `active=false`; Datenstrom endet nach Bestätigung beziehungsweise spätestens nach Lease-Ablauf.

- State4-Schalter aktivieren/deaktivieren: vollständige sichtbare Liste startet und stoppt ohne persistente Setting-Änderung.
- Fahrfähigkeit aktualisieren: publish_now auf gps_state/state1/request und neuer Snapshot.
- Alle vier Restart-Reset-Modi testen; validation, status und last sichtbar prüfen.
- Logging Start/Stop/Cancel testen und sicherstellen, dass Buttonzustände ausschließlich dem Backendstatus folgen.
- Session- und persistente Settings getrennt schreiben und rejected-Felder aus validation anzeigen.

11. Commit-Vorschlag

feat(gps-state): align app with canonical state1-state4 MQTT contract

- remove legacy state0 MQTT usage
- activate state2-state4 through renewable UI leases
- add state2 satellite payload and canonical requests
- support hardware_after_shutdown restart mode and request IDs
- align GPS logging and settings runtime behavior with ROS contract